PREDICTIVE ANALYSIS ON SYNTHETIC STOCK DATA

Oindrila Chakraborty

2026-04-09

```
#install.packages("car")
#install.packages("leaps")
#install.packages("glmnet")
#install.packages("caret")
#install.packages("regclass")
#install.packages("class")
#install.packages("pROC")
library(leaps)

## Warning: package 'leaps' was built under R version 4.3.3

library(glmnet)

## Warning: package 'glmnet' was built under R version 4.3.3

## Loading required package: Matrix

## Loaded glmnet 4.1-8

library(caret)

## Warning: package 'caret' was built under R version 4.3.3

## Loading required package: ggplot2

## Loading required package: lattice

library(regclass)

## Loading required package: bestglm

## Warning: package 'bestglm' was built under R version 4.3.3

## Loading required package: VGAM

## Warning: package 'VGAM' was built under R version 4.3.3

## Loading required package: stats4

## Loading required package: splines

##
## Attaching package: 'VGAM'
```

```
## The following object is masked from 'package:caret':
##
##     predictors

## Loading required package: rpart

## Loading required package: randomForest

## Warning: package 'randomForest' was built under R version 4.3.3

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

## Important regclass change from 1.3:
## All functions that had a . in the name now have an _
## all.correlations -> all_correlations, cor.demo -> cor_demo, etc.

##
## Attaching package: 'regclass'

## The following object is masked from 'package:lattice':
##
##     qq

library(class)

## Warning: package 'class' was built under R version 4.3.3

library(pROC)

## Warning: package 'pROC' was built under R version 4.3.3

## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var

#library(car)
data=read.csv("C:/Users/OINDRILA CHAKRABORTY/Desktop/synthetic_stock_data
(1).csv")
```

```r
# Basic Exploration
dim(data)
```

```
## [1] 1000   14
```

```r
str(data)
```

```
## 'data.frame':    1000 obs. of  14 variables:
##  $ Date           : chr  "01-01-2022" "02-01-2022" "03-01-2022" "04-01-
2022" ...
##  $ Company        : chr  "Uber" "Tesla" "Panasonic" "Tencent" ...
##  $ Sector         : chr  "Technology" "Automotive" "Finance" "Automotive"
...
##  $ Open           : num  100 100.1 99.9 98.9 98.4 ...
##  $ High           : num  101 102 102 101 100 ...
##  $ Low            : num  97.5 97.2 98.1 97 96.2 ...
##  $ Close          : num  100 100.1 99.9 98.9 98.4 ...
##  $ Volume         : int  171958 196867 181932 153694 169879 160268 104886
187337 137498 162727 ...
##  $ Market_Cap     : num  5.16e+11 9.76e+11 4.60e+11 5.58e+11 8.61e+11 ...
##  $ PE_Ratio       : num  24.3 18.6 10.7 14.6 37.5 ...
##  $ Dividend_Yield : num  0.163 0.289 2.222 1.378 3.11 ...
##  $ Volatility     : num  0.0475 0.0225 0.02 0.0362 0.0348 ...
##  $ Sentiment_Score: num  0.939 0.469 0.399 0.706 -0.768 ...
##  $ Trend          : chr  "Bearish" "Bearish" "Bullish" "Stable" ...
```

```r
summary(data)
```

```
##      Date              Company             Sector               Open
##  Length:1000        Length:1000        Length:1000        Min.   : 82.24
##  Class :character   Class :character   Class :character   1st Qu.: 99.75
##  Mode  :character   Mode  :character   Mode  :character   Median :113.82
##                                                           Mean   :110.13
##                                                           3rd Qu.:118.82
##                                                           Max.   :128.99
##       High             Low              Close            Volume
##  Min.   : 83.27   Min.   : 79.88   Min.   : 82.24   Min.   : 50126
##  1st Qu.:101.96   1st Qu.: 97.96   1st Qu.: 99.75   1st Qu.: 87993
##  Median :116.04   Median :111.39   Median :113.82   Median :127540
##  Mean   :112.34   Mean   :107.93   Mean   :110.13   Mean   :126117
##  3rd Qu.:121.28   3rd Qu.:116.60   3rd Qu.:118.82   3rd Qu.:165310
##  Max.   :132.20   Max.   :126.69   Max.   :128.99   Max.   :199849
##    Market_Cap           PE_Ratio      Dividend_Yield      Volatility
##  Min.   :1.237e+09   Min.   : 5.007   Min.   :0.01009   Min.   :0.01001
##  1st Qu.:2.563e+11   1st Qu.:13.784   1st Qu.:1.35742   1st Qu.:0.01929
##  Median :5.154e+11   Median :23.453   Median :2.60287   Median :0.02879
##  Mean   :5.039e+11   Mean   :22.779   Mean   :2.55085   Mean   :0.02935
##  3rd Qu.:7.484e+11   3rd Qu.:31.539   3rd Qu.:3.77382   3rd Qu.:0.03961
##  Max.   :9.994e+11   Max.   :39.990   Max.   :4.99772   Max.   :0.04990
##  Sentiment_Score       Trend
##  Min.   :-0.99585   Length:1000
```

```
##  1st Qu.:-0.51570   Class :character
##  Median :-0.02448   Mode  :character
##  Mean   :-0.02240
##  3rd Qu.: 0.46888
##  Max.   : 0.99810
```

```
cat("----- STRUCTURE -----\n")
```

```
## ----- STRUCTURE -----
```

```
str(data)
```

```
## 'data.frame':    1000 obs. of  14 variables:
##  $ Date           : chr  "01-01-2022" "02-01-2022" "03-01-2022" "04-01-
2022" ...
##  $ Company        : chr  "Uber" "Tesla" "Panasonic" "Tencent" ...
##  $ Sector         : chr  "Technology" "Automotive" "Finance" "Automotive"
...
##  $ Open           : num  100 100.1 99.9 98.9 98.4 ...
##  $ High           : num  101 102 102 101 100 ...
##  $ Low            : num  97.5 97.2 98.1 97 96.2 ...
##  $ Close          : num  100 100.1 99.9 98.9 98.4 ...
##  $ Volume         : int  171958 196867 181932 153694 169879 160268 104886
187337 137498 162727 ...
##  $ Market_Cap     : num  5.16e+11 9.76e+11 4.60e+11 5.58e+11 8.61e+11 ...
##  $ PE_Ratio       : num  24.3 18.6 10.7 14.6 37.5 ...
##  $ Dividend_Yield : num  0.163 0.289 2.222 1.378 3.11 ...
##  $ Volatility     : num  0.0475 0.0225 0.02 0.0362 0.0348 ...
##  $ Sentiment_Score: num  0.939 0.469 0.399 0.706 -0.768 ...
##  $ Trend          : chr  "Bearish" "Bearish" "Bullish" "Stable" ...
```

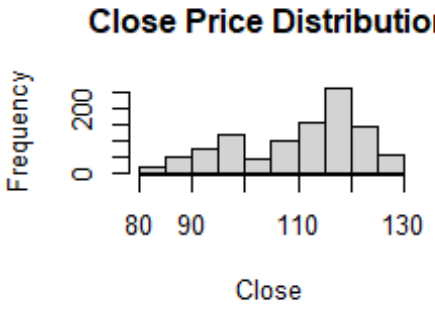
```
cat("\n----- SUMMARY -----\n")
```

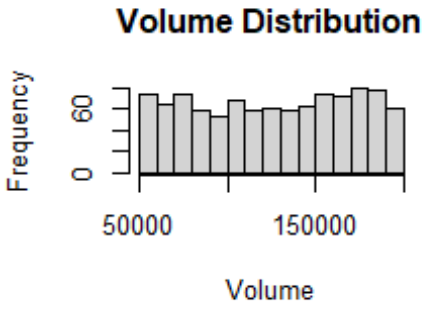
```
##
## ----- SUMMARY -----
```

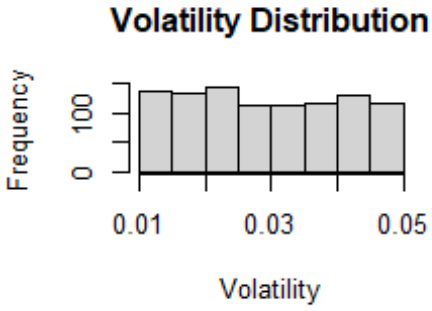
```
print(summary(data))
```

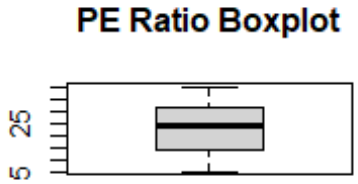
```
##      Date             Company             Sector               Open
##  Length:1000        Length:1000        Length:1000        Min.   : 82.24
##  Class :character   Class :character   Class :character   1st Qu.: 99.75
##  Mode  :character   Mode  :character   Mode  :character   Median :113.82
##                                                           Mean   :110.13
##                                                           3rd Qu.:118.82
##                                                           Max.   :128.99
##       High             Low              Close            Volume
##  Min.   : 83.27   Min.   : 79.88   Min.   : 82.24   Min.   : 50126
##  1st Qu.:101.96   1st Qu.: 97.96   1st Qu.: 99.75   1st Qu.: 87993
##  Median :116.04   Median :111.39   Median :113.82   Median :127540
##  Mean   :112.34   Mean   :107.93   Mean   :110.13   Mean   :126117
##  3rd Qu.:121.28   3rd Qu.:116.60   3rd Qu.:118.82   3rd Qu.:165310
##  Max.   :132.20   Max.   :126.69   Max.   :128.99   Max.   :199849
```

```
##    Market_Cap           PE_Ratio      Dividend_Yield      Volatility
##  Min.   :1.237e+09   Min.   : 5.007   Min.   :0.01009   Min.   :0.01001
##  1st Qu.:2.563e+11   1st Qu.:13.784   1st Qu.:1.35742   1st Qu.:0.01929
##  Median :5.154e+11   Median :23.453   Median :2.60287   Median :0.02879
##  Mean   :5.039e+11   Mean   :22.779   Mean   :2.55085   Mean   :0.02935
##  3rd Qu.:7.484e+11   3rd Qu.:31.539   3rd Qu.:3.77382   3rd Qu.:0.03961
##  Max.   :9.994e+11   Max.   :39.990   Max.   :4.99772   Max.   :0.04990
##  Sentiment_Score       Trend
##  Min.   :-0.99585   Length:1000
##  1st Qu.:-0.51570   Class :character
##  Median :-0.02448   Mode  :character
##  Mean   :-0.02240
##  3rd Qu.: 0.46888
##  Max.   : 0.99810

data$Date <- as.Date(data$Date, format="%d-%m-%Y")
data$Company <- as.factor(data$Company)
data$Sector <- as.factor(data$Sector)
data$Trend <- as.factor(data$Trend)

#missing values
cat("\n----- MISSING VALUES -----\n")

##
## ----- MISSING VALUES -----

print(colSums(is.na(data)))

##            Date         Company          Sector            Open
High
##               0               0               0               0
0
##             Low           Close          Volume      Market_Cap
PE_Ratio
##               0               0               0               0
0
##  Dividend_Yield      Volatility Sentiment_Score           Trend
##               0               0               0               0

#univariate analysis
par(mfrow=c(2,2))

hist(data$Close, main="Close Price Distribution", xlab="Close")
hist(data$Volume, main="Volume Distribution", xlab="Volume")
hist(data$Volatility, main="Volatility Distribution", xlab="Volatility")
boxplot(data$PE_Ratio, main="PE Ratio Boxplot")
```

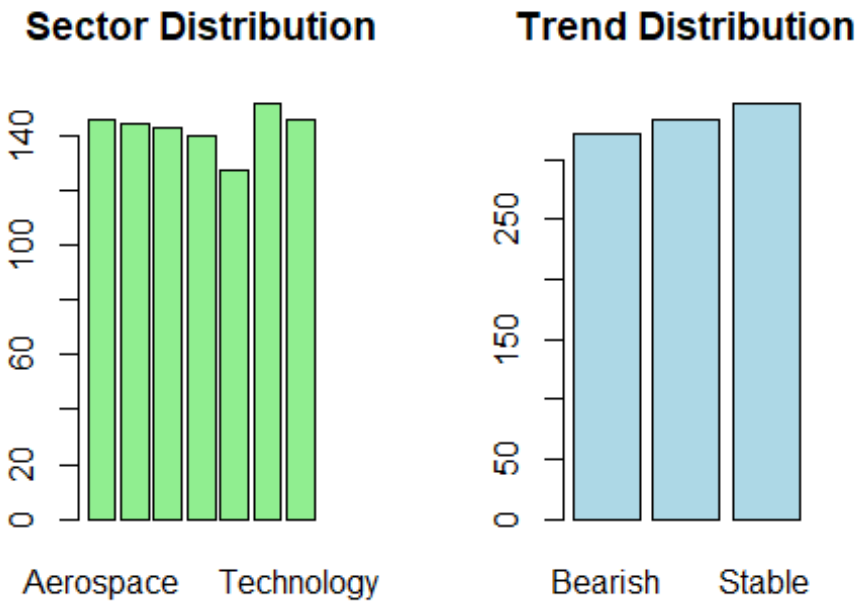




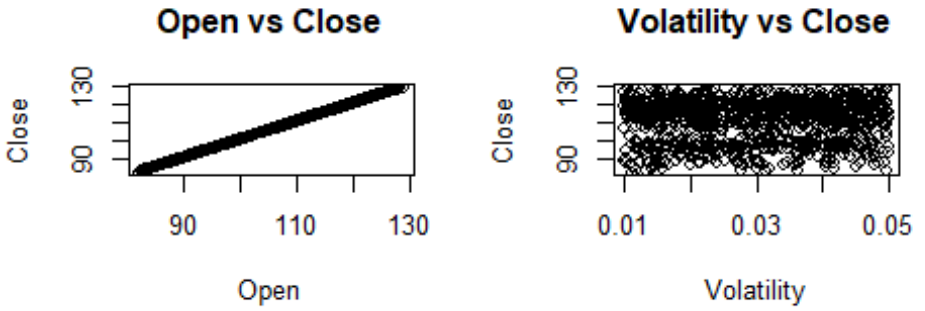


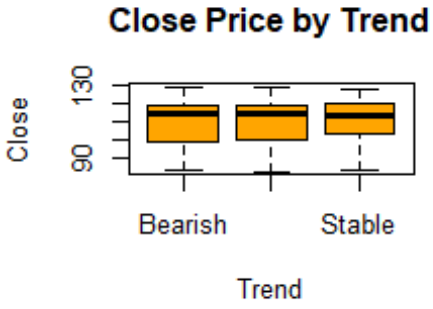


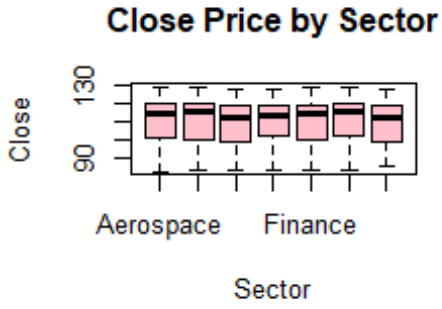

```
#categorical analysis
cat("\n----- CATEGORICAL TABLES -----\n")

##
## ----- CATEGORICAL TABLES -----

print(table(data$Sector))

##
##      Aerospace     Automotive Consumer Goods         Energy        Finance
##            146            145            143            140            128
##     Healthcare     Technology
##            152            146

print(table(data$Trend))

##
## Bearish Bullish  Stable
##     321     333     346

par(mfrow=c(1,2))
barplot(table(data$Sector), col="lightgreen", main="Sector Distribution")
barplot(table(data$Trend), col="lightblue", main="Trend Distribution")
```




```r
#bivariate analysis
par(mfrow=c(2,2))

plot(data$Open, data$Close,
     main="Open vs Close",
     xlab="Open", ylab="Close")

plot(data$Volatility, data$Close,
     main="Volatility vs Close",
     xlab="Volatility", ylab="Close")

boxplot(Close ~ Trend, data=data,
        main="Close Price by Trend",
        col="orange")

boxplot(Close ~ Sector, data=data,
        main="Close Price by Sector",
        col="pink")
```







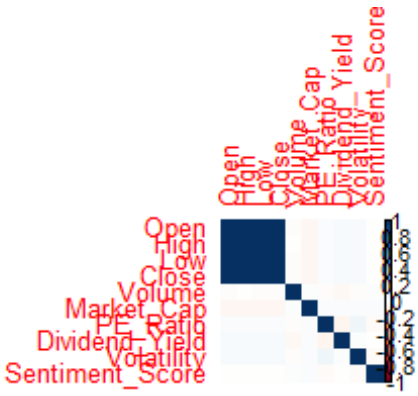

```r
#correlation analysis
numeric_data <- data[, sapply(data, is.numeric)]

cor_matrix <- cor(numeric_data, use="complete.obs")

cat("\n----- CORRELATION MATRIX -----\n")
```

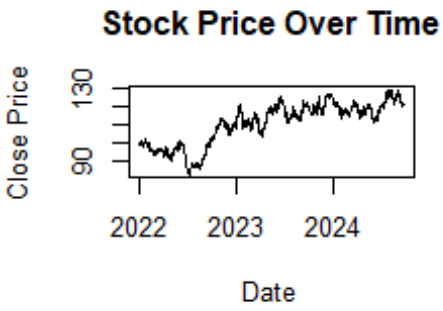
```
##
## ----- CORRELATION MATRIX -----
```

```r
print(cor_matrix)
```

```
##                         Open         High          Low        Close
## Open             1.000000000  0.998483659  0.998411871  1.000000000
## High             0.998483659  1.000000000  0.997164046  0.998483659
## Low              0.998411871  0.997164046  1.000000000  0.998411871
## Close            1.000000000  0.998483659  0.998411871  1.000000000
## Volume          -0.012947873 -0.009041004 -0.016789356 -0.012947873
## Market_Cap      -0.043008058 -0.043719838 -0.044088985 -0.043008058
## PE_Ratio         0.027296874  0.026743446  0.026786650  0.027296874
## Dividend_Yield   0.033202909  0.033306024  0.031150042  0.033202909
## Volatility       0.037692271  0.035231220  0.035145069  0.037692271
## Sentiment_Score -0.007439058 -0.005486858 -0.004524029 -0.007439058
##                       Volume   Market_Cap    PE_Ratio Dividend_Yield
Volatility
## Open            -0.012947873 -0.04300806  0.02729687    0.033202909
0.03769227
```

```
## High            -0.009041004 -0.04371984  0.02674345    0.033306024
0.03523122
## Low             -0.016789356 -0.04408898  0.02678665    0.031150042
0.03514507
## Close           -0.012947873 -0.04300806  0.02729687    0.033202909
0.03769227
## Volume           1.000000000  0.01832040 -0.01808314   -0.034696246 -
0.01621291
## Market_Cap       0.018320397  1.00000000  0.02275416    0.025674589
0.02375447
## PE_Ratio        -0.018083139  0.02275416  1.00000000   -0.056375956
0.06605896
## Dividend_Yield  -0.034696246  0.02567459 -0.05637596    1.000000000
0.01895221
## Volatility      -0.016212910  0.02375447  0.06605896    0.018952209
1.00000000
## Sentiment_Score  0.013239281  0.02405725 -0.02488441   -0.004240946 -
0.02076919
##                 Sentiment_Score
## Open               -0.007439058
## High               -0.005486858
## Low                -0.004524029
## Close              -0.007439058
## Volume              0.013239281
## Market_Cap          0.024057247
## PE_Ratio           -0.024884406
## Dividend_Yield     -0.004240946
## Volatility         -0.020769189
## Sentiment_Score     1.000000000

# Install if needed
if(!require(corrplot)) install.packages("corrplot")

## Loading required package: corrplot

## Warning: package 'corrplot' was built under R version 4.3.3

## corrplot 0.95 loaded

library(corrplot)

corrplot(cor_matrix, method="color")

#time series plot
plot(data$Date, data$Close, type="l",
     main="Stock Price Over Time",
     xlab="Date", ylab="Close Price")
```





1. Data Overview

The dataset contains synthetic stock market data with variables such as price, volume, volatility, and financial indicators.Both numerical and categorical variables are present, making it suitable for statistical analysis and modeling.

2. Missing Value Analysis No significant missing values were observed in the dataset.The dataset is clean and suitable for further analysis.

3. Univariate Analysis The closing price shows a moderate spread, indicating variability in stock values.Volume distribution is slightly skewed, suggesting uneven trading activity.Volatility indicates fluctuations in stock prices over time.Some variables (like PE Ratio) show presence of outliers, indicating extreme values.

4. Categorical Analysis The dataset includes multiple sectors and trend categories.Certain sectors have higher representation, indicating dominance in the dataset.Trend distribution shows variation among uptrend, downtrend, and stable categories.

5. Bivariate Analysis A strong positive relationship is observed between open and close prices, indicating consistency in stock movement.Volatility vs close price shows that higher volatility is associated with larger price fluctuations.Boxplots indicate that stock prices vary across different trend categories and sectors.

6. Correlation Analysis High correlation exists among open, high, low, and close prices, which is expected in stock data.Other variables show moderate to low correlation, indicating independent effects.No severe multicollinearity issues observed beyond price variables.

7. Time Series Analysis Stock prices exhibit fluctuating trends over time, reflecting realistic market behavior.Periods of increase and decrease indicate dynamic movement in synthetic data.

```
#linear regression model
model=lm(Close~ ., data = data)
m=summary(model)
```

```
## Warning in summary.lm(model): essentially perfect fit: summary may be
## unreliable
```

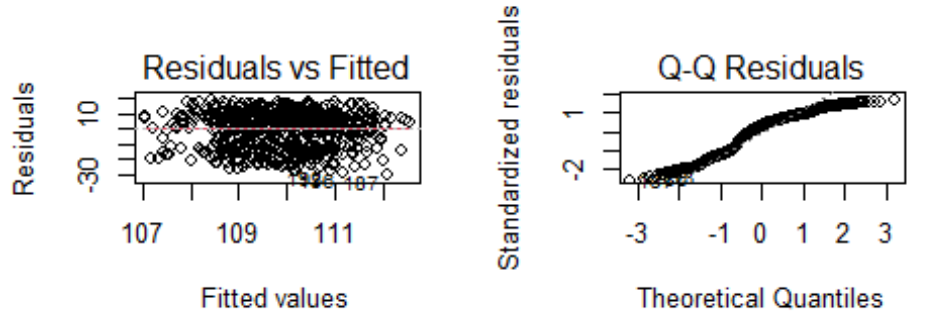
```
# TRAIN-TEST SPLIT
set.seed(123)
train_idx=sample(1:nrow(data), 0.7*nrow(data))
train=data[train_idx, ]
test=data[-train_idx, ]
#head(train)
#head(test)
# MULTIPLE LINEAR REGRESSION
# Fit the model using specific numeric columns
lm_model <- lm(Close ~ Volume + PE_Ratio + Dividend_Yield +
                 Volatility + Sentiment_Score, data = train)
cat("----- MODEL SUMMARY -----\n")
```

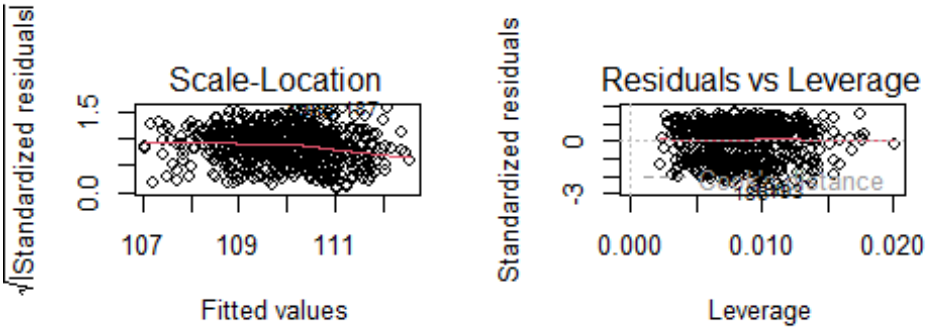
```
## ----- MODEL SUMMARY -----
```

```
summary(lm_model)
```

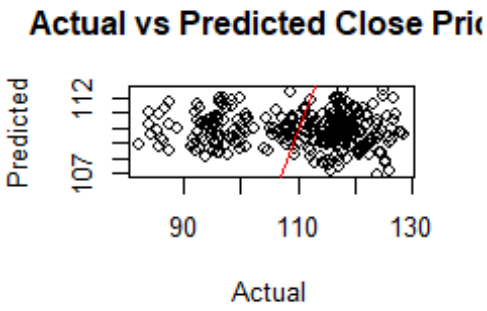
```
##
## Call:
## lm(formula = Close ~ Volume + PE_Ratio + Dividend_Yield + Volatility +
##     Sentiment_Score, data = train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -29.529 -10.481   3.223   8.810  18.618
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.080e+02  2.180e+00  49.550   <2e-16 ***
## Volume          -6.129e-06  9.888e-06  -0.620    0.536
## PE_Ratio         6.020e-02  4.366e-02   1.379    0.168
## Dividend_Yield   4.232e-01  3.118e-01   1.357    0.175
## Volatility       7.735e+00  3.764e+01   0.206    0.837
## Sentiment_Score  9.524e-01  7.757e-01   1.228    0.220
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11.56 on 694 degrees of freedom
## Multiple R-squared:  0.00783,    Adjusted R-squared:  0.0006815
## F-statistic: 1.095 on 5 and 694 DF,  p-value: 0.3616
```

```r
# Prediction (THIS IS MISSING IN YOUR ERROR)
lm_pred <- predict(lm_model, newdata = test)
# MSE
lm_mse <- mean((test$Close - lm_pred)^2)
# MSE
mse <- mean((test$Close - lm_pred)^2)
# RMSE
rmse <- sqrt(mse)
# MAE
mae <- mean(abs(test$Close - lm_pred))
# R-squared (test)
ss_total <- sum((test$Close - mean(test$Close))^2)
ss_res <- sum((test$Close - lm_pred)^2)
r2 <- 1 - (ss_res / ss_total)

cat("\n----- MODEL PERFORMANCE -----\n")

##
## ----- MODEL PERFORMANCE -----

cat("MSE:", mse, "\n")

## MSE: 126.8975

cat("RMSE:", rmse, "\n")

## RMSE: 11.26488

cat("MAE:", mae, "\n")

## MAE: 9.650302

cat("R-squared:", r2, "\n")

## R-squared: -0.02851896

cat("\n----- VIF VALUES -----\n")

##
## ----- VIF VALUES -----

VIF(lm_model)

##          Volume        PE_Ratio  Dividend_Yield      Volatility
Sentiment_Score
##        1.008160        1.010547        1.004361        1.008908
1.003725

#RESIDUAL ANALYSIS
par(mfrow=c(2,2))
plot(lm_model)  # Diagnostic plots
```



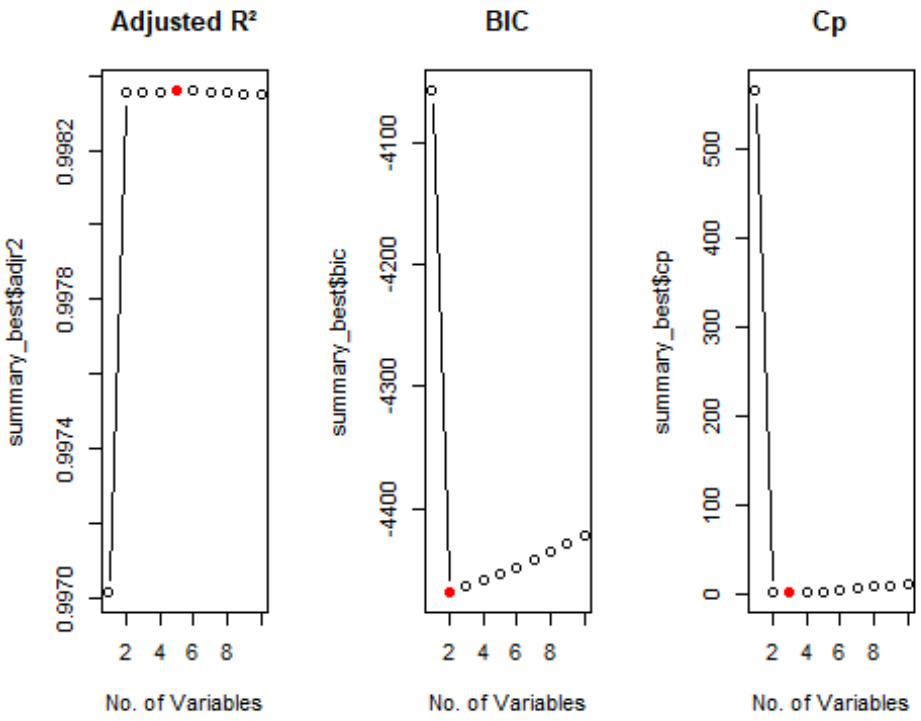



```r
#ACTUAL vs PREDICTED PLOT
plot(test$Close, lm_pred,
     main="Actual vs Predicted Close Price",
     xlab="Actual", ylab="Predicted")
abline(0,1,col="red")
```




```r
#if(!require(leaps)) install.packages("leaps")
#library(leaps)

#subset selection
subset_fit <- regsubsets(Close ~ . - Date - Company - Sector - Open,
                         data = train,
                         nvmax = 10)
summary_best <- summary(subset_fit)
best_adjr2 <- which.max(summary_best$adjr2)
best_bic   <- which.min(summary_best$bic)
best_cp    <- which.min(summary_best$cp)

cat("Best model by Adjusted R²:", best_adjr2, "\n")

## Best model by Adjusted R²: 5

cat("Best model by BIC:", best_bic, "\n")

## Best model by BIC: 2

cat("Best model by Cp:", best_cp, "\n")

## Best model by Cp: 3

par(mfrow=c(1,3))
plot(summary_best$adjr2, type="b", main="Adjusted R²", xlab="No. of
Variables")
points(best_adjr2, summary_best$adjr2[best_adjr2], col="red", pch=19)
```

```r
plot(summary_best$bic, type="b", main="BIC", xlab="No. of Variables")
points(best_bic, summary_best$bic[best_bic], col="red", pch=19)

plot(summary_best$cp, type="b", main="Cp", xlab="No. of Variables")
points(best_cp, summary_best$cp[best_cp], col="red", pch=19)
```




```r
predict_regsubsets <- function(object, newdata, id){
  form <- as.formula(object$call[[2]])
  mat  <- model.matrix(form, newdata)
  coefi <- coef(object, id = id)
  xvars <- names(coefi)
  mat[, xvars] %*% coefi
}

subset_pred <- predict_regsubsets(subset_fit, test, best_bic)

mse  <- mean((test$Close - drop(subset_pred))^2)
rmse <- sqrt(mse)
mae  <- mean(abs(test$Close - drop(subset_pred)))

# R-squared
ss_total <- sum((test$Close - mean(test$Close))^2)
ss_res   <- sum((test$Close - subset_pred)^2)
r2       <- 1 - (ss_res / ss_total)
```

```r
cat("\n----- SUBSET MODEL PERFORMANCE -----\n")
```

```
##
## ----- SUBSET MODEL PERFORMANCE -----
```

```r
cat("MSE:", mse, "\n")
```

```
## MSE: 0.2223449
```

```r
cat("RMSE:", rmse, "\n")
```

```
## RMSE: 0.4715346
```

```r
cat("MAE:", mae, "\n")
```

```
## MAE: 0.3806359
```

```r
cat("R-squared:", r2, "\n")
```

```
## R-squared: 0.9981979
```

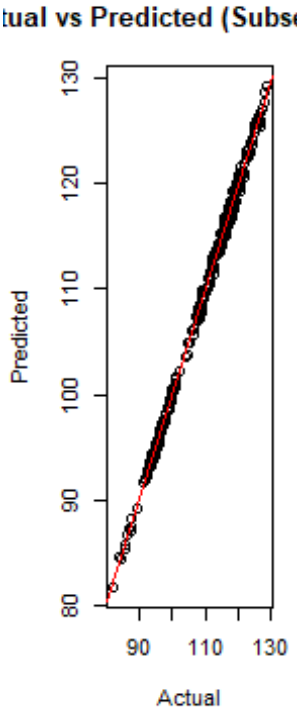
```r
cat("\n----- BEST MODEL COEFFICIENTS (BIC) -----\n")
```

```
##
## ----- BEST MODEL COEFFICIENTS (BIC) -----
```

```r
print(coef(subset_fit, best_bic))
```

```
## (Intercept)        High         Low
##   0.3762149   0.4912986   0.5055587
```

```r
plot(test$Close, subset_pred,
     main="Actual vs Predicted (Subset Model)",
     xlab="Actual", ylab="Predicted")
abline(0,1,col="red")
```




```r
# RIDGE & LASSO
library(glmnet)
# Matrix creation
x = model.matrix(Close ~ . - Date - Company - Sector - Open, data = data)[, -
1]
y = data$Close
x_train = x[train_idx, ]
y_train = y[train_idx]
x_test  = x[-train_idx, ]
y_test  = y[-train_idx]

cv_ridge = cv.glmnet(x_train, y_train, alpha = 0)
best_lambda_ridge = cv_ridge$lambda.min
ridge_pred = predict(cv_ridge, s = "lambda.min", newx = x_test)
ridge_mse  = mean((y_test - ridge_pred)^2)
ridge_rmse = sqrt(ridge_mse)

cat("\n--- RIDGE RESULTS ---\n")

##
## --- RIDGE RESULTS ---

cat("Best Lambda:", best_lambda_ridge, "\n")

## Best Lambda: 1.154336

cat("RMSE:", ridge_rmse, "\n")
```

```
## RMSE: 0.7150273
```

```
cv_lasso = cv.glmnet(x_train, y_train, alpha = 1)

best_lambda_lasso = cv_lasso$lambda.min

lasso_pred = predict(cv_lasso, s = "lambda.min", newx = x_test)

lasso_mse  = mean((y_test - lasso_pred)^2)
lasso_rmse = sqrt(lasso_mse)

cat("\n--- LASSO RESULTS ---\n")
```

```
##
## --- LASSO RESULTS ---
```

```
cat("Best Lambda:", best_lambda_lasso, "\n")
```

```
## Best Lambda: 0.07594759
```

```
cat("RMSE:", lasso_rmse, "\n")
```

```
## RMSE: 0.4779644
```

```
# Important: Selected variables in LASSO
cat("\n--- LASSO SELECTED VARIABLES ---\n")
```

```
##
## --- LASSO SELECTED VARIABLES ---
```

```
print(coef(cv_lasso, s = "lambda.min"))
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
##                        s1
## (Intercept)     1.0900736
## High            0.5400469
## Low             0.4481844
## Volume          .
## Market_Cap      .
## PE_Ratio        .
## Dividend_Yield  .
## Volatility      .
## Sentiment_Score .
## TrendBullish    .
## TrendStable     .
```

```
# KNN REGRESSION
library(caret)

set.seed(123)
```

```r
control <- trainControl(method="cv", number=5)

knn_tuned <- train(Close ~ . - Date - Company - Sector - Open,
                   data = train,
                   method = "knn",
                   trControl = control,
                   tuneLength = 10)

knn_pred <- predict(knn_tuned, test)

knn_mse  <- mean((test$Close - knn_pred)^2)
knn_rmse <- sqrt(knn_mse)

cat("\n--- KNN RESULTS ---\n")
```

```
##
## --- KNN RESULTS ---
```

```r
print(knn_tuned$bestTune)
```

```
##     k
## 10 23
```

```r
cat("RMSE:", knn_rmse, "\n")
```

```
## RMSE: 11.245
```

```r
# LOGISTIC REGRESSION
library(pROC)

train$IsBullish = ifelse(train$Trend == "Bullish", 1, 0)
test$IsBullish  = ifelse(test$Trend == "Bullish", 1, 0)

log_model = glm(IsBullish ~ Volume + PE_Ratio + Volatility + Sentiment_Score,
                data = train, family = binomial)

# Summary
summary(log_model)
```

```
##
## Call:
## glm(formula = IsBullish ~ Volume + PE_Ratio + Volatility +
Sentiment_Score,
##     family = binomial, data = train)
##
## Coefficients:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -1.042e+00  3.682e-01  -2.830  0.00465 **
## Volume           1.037e-06  1.797e-06   0.577  0.56372
## PE_Ratio         7.560e-03  7.932e-03   0.953  0.34057
```

```
## Volatility       3.865e+00  6.832e+00   0.566  0.57158
## Sentiment_Score  4.652e-02  1.410e-01   0.330  0.74142
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 905.18  on 699  degrees of freedom
## Residual deviance: 903.51  on 695  degrees of freedom
## AIC: 913.51
##
## Number of Fisher Scoring iterations: 4
```

```
# Prediction
test_prob = predict(log_model, newdata = test, type = "response")

# ROC Curve
roc_obj = roc(test$IsBullish, test_prob)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls > cases
```

```
# Best cutoff
best_cut = coords(roc_obj, "best", ret="threshold")

# Final classification
pred_class = ifelse(test_prob > as.numeric(best_cut), 1, 0)

# Confusion matrix
conf_matrix = table(Predicted = pred_class, Actual = test$IsBullish)

# Accuracy
accuracy = mean(pred_class == test$IsBullish)

cat("\n--- LOGISTIC RESULTS ---\n")
```

```
##
## --- LOGISTIC RESULTS ---
```

```
print(conf_matrix)
```

```
##          Actual
## Predicted   0   1
##         0  79  42
##         1 132  47
```

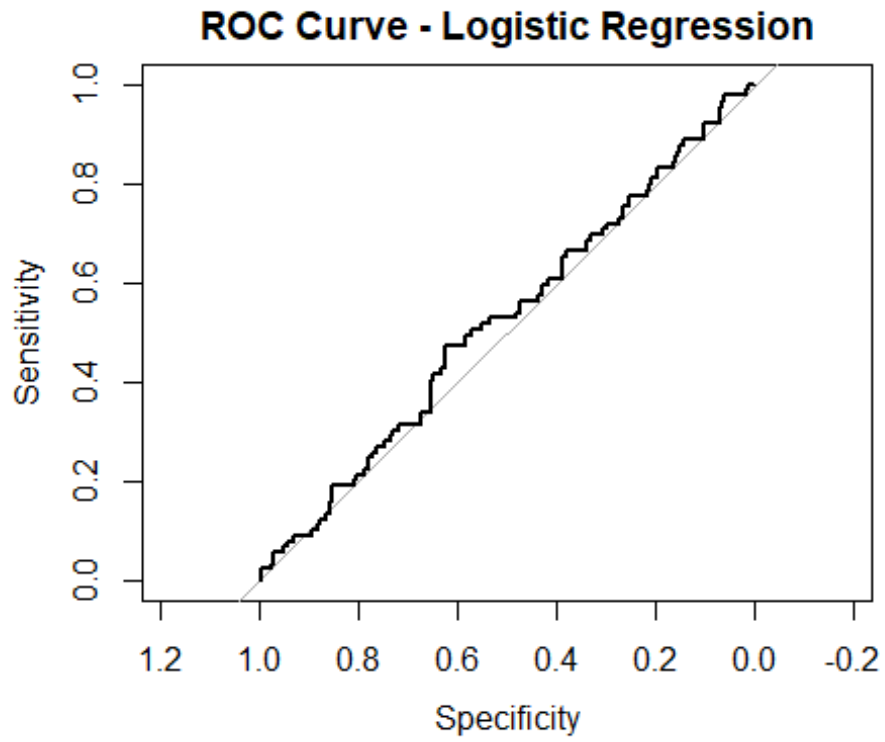
```
cat("Accuracy:", accuracy, "\n")
```

```
## Accuracy: 0.42
```

```r
cat("AUC:", auc(roc_obj), "\n")
```

```
## AUC: 0.5210075
```

```r
plot(roc_obj, main = "ROC Curve - Logistic Regression")
```




```r
auc(roc_obj)
```

```
## Area under the curve: 0.521
```

```r
#LDA
library(MASS)
lda_model <- lda(Trend ~ Volume + PE_Ratio + Volatility + Sentiment_Score,
data = train)
lda_pred <- predict(lda_model, test)
# Predicted class
lda_class <- lda_pred$class
# Confusion matrix
table(Predicted = lda_class, Actual = test$Trend)
```

```
##           Actual
## Predicted Bearish Bullish Stable
##   Bearish      13      12     10
##   Bullish      53      39     46
##   Stable       44      38     45
```

```r
# Accuracy
lda_accuracy <- mean(lda_class == test$Trend)
cat("LDA Accuracy:", lda_accuracy)
```

```
## LDA Accuracy: 0.3233333
```

```r
# KNN CLASSIFICATION
library(class)

# Target variables
train_y_cat = train$Trend
test_y_cat  = test$Trend

# KNN Classification
knn_class_pred = knn(train = x_train, test = x_test, cl = train_y_cat, k = 5)

# Confusion Matrix
conf_matrix_knn = table(Predicted = knn_class_pred, Actual = test_y_cat)

# Accuracy & Error
knn_accuracy = mean(knn_class_pred == test_y_cat)
knn_error = mean(knn_class_pred != test_y_cat)

cat("\n--- KNN CLASSIFICATION RESULTS ---\n")
```

```
##
## --- KNN CLASSIFICATION RESULTS ---
```

```r
print(conf_matrix_knn)
```

```
##           Actual
## Predicted Bearish Bullish Stable
##   Bearish      27      21     31
##   Bullish      39      34     33
##   Stable       44      34     37
```

```r
cat("Accuracy:", knn_accuracy, "\n")
```

```
## Accuracy: 0.3266667
```

```r
cat("Error:", knn_error, "\n")
```

```
## Error: 0.6733333
```

```r
# STANDARDIZE METRICS (RMSE)
lm_rmse     = sqrt(lm_mse)
subset_rmse = sqrt(mse)
ridge_rmse  = sqrt(ridge_mse)
lasso_rmse  = sqrt(lasso_mse)
knn_rmse    = sqrt(knn_mse)
```

```r
# FINAL REGRESSION COMPARISON
results = data.frame(
  Model = c("Linear Regression",
            "Subset Selection",
            "Ridge Regression",
            "LASSO Regression",
            "KNN Regression"),
  RMSE = c(lm_rmse, subset_rmse, ridge_rmse, lasso_rmse, knn_rmse)
)

cat("\n--- REGRESSION MODEL COMPARISON ---\n")
```

```
##
## --- REGRESSION MODEL COMPARISON ---
```

```r
print(results)
```

```
##               Model       RMSE
## 1 Linear Regression 11.2648793
## 2  Subset Selection  0.4715346
## 3  Ridge Regression  0.7150273
## 4  LASSO Regression  0.4779644
## 5    KNN Regression 11.2450043
```

```r
# CLASSIFICATION COMPARISON
class_results = data.frame(
  Model = c("Logistic Regression", "KNN Classification"),
  Accuracy = c(accuracy, knn_accuracy)
)

cat("\n--- CLASSIFICATION MODEL COMPARISON ---\n")
```

```
##
## --- CLASSIFICATION MODEL COMPARISON ---
```

```r
print(class_results)
```

```
##                 Model  Accuracy
## 1 Logistic Regression 0.4200000
## 2  KNN Classification 0.3266667
```

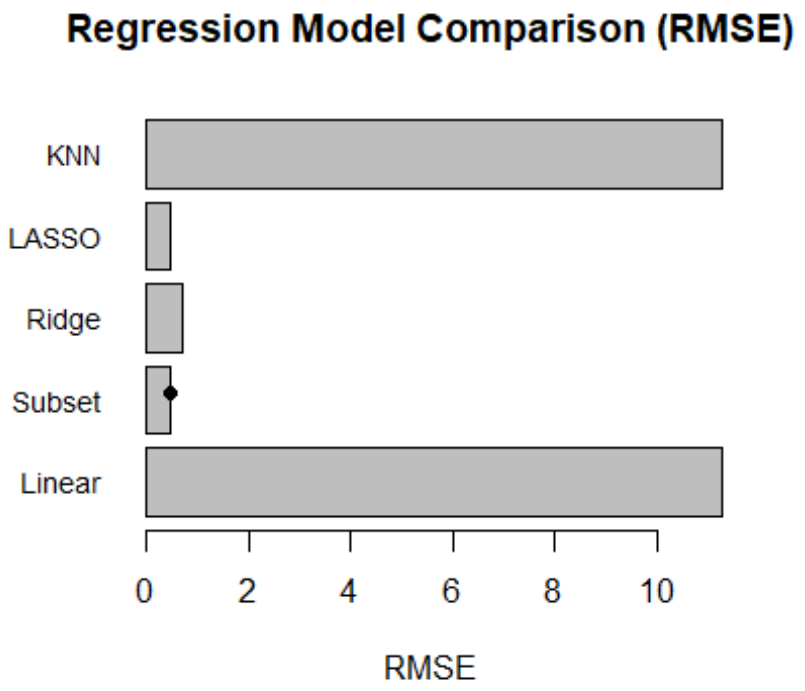
```r
# BEST MODEL IDENTIFICATION
best_model = results$Model[which.min(results$RMSE)]

cat("\n--- BEST REGRESSION MODEL ---\n")
```

```
##
## --- BEST REGRESSION MODEL ---
```

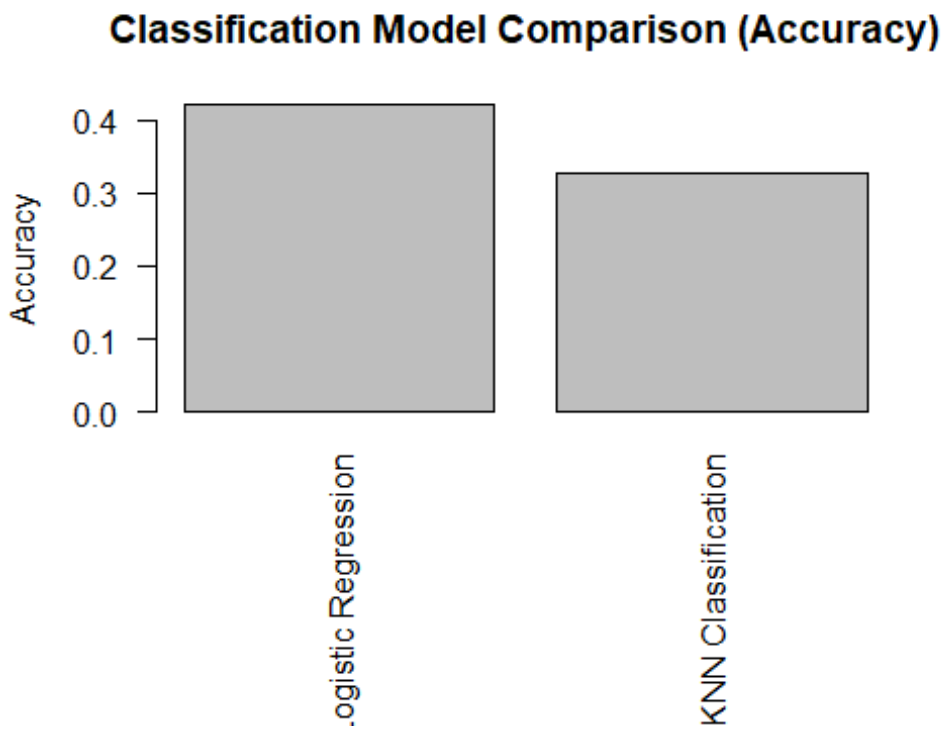
```r
cat("Best Model based on RMSE:", best_model, "\n")
```

```
## Best Model based on RMSE: Subset Selection
```

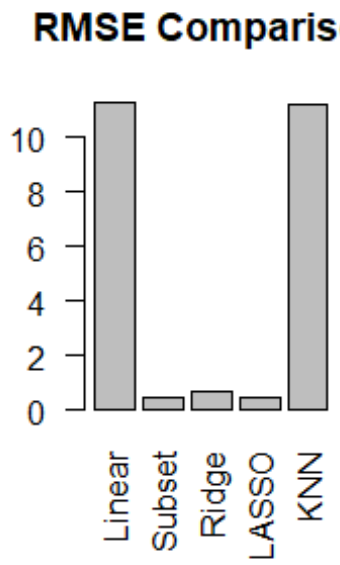
```r
results$Model <- c("Linear", "Subset", "Ridge", "LASSO", "KNN")
par(mar = c(5, 8, 4, 2))
barplot(results$RMSE,
        names.arg = results$Model,
        horiz = TRUE,
        cex.names = 0.9,
        main = "Regression Model Comparison (RMSE)",
        xlab = "RMSE",
        las = 1)

# Highlight best model
best_index = which.min(results$RMSE)
points(results$RMSE[best_index], best_index, pch = 19)
```

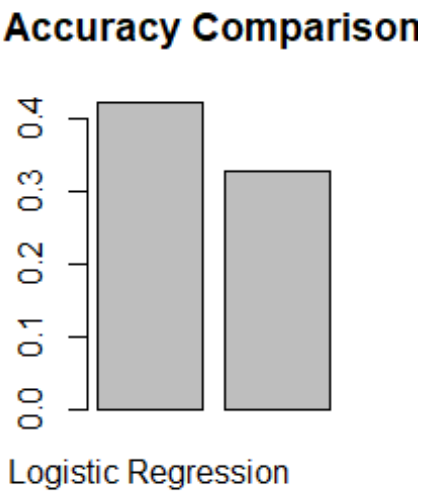



```r
par(mar = c(8, 4, 4, 2))
barplot(class_results$Accuracy,
        names.arg = class_results$Model,
        las=2,
        main = "Classification Model Comparison (Accuracy)",
        ylab = "Accuracy")
```




```r
par(mfrow = c(1,2))
# Regression Plot
barplot(results$RMSE,
        names.arg = results$Model,
        main = "RMSE Comparison",
        las = 2)
# Classification Plot
barplot(class_results$Accuracy,
        names.arg = class_results$Model,
        main = "Accuracy Comparison")
```






```
# Reset layout
par(mfrow = c(1,1))
```

Project Report: Predictive Modeling and Analysis of Stock Market Trends

1. Project Overview

The objective of this project was to analyze a synthetic stock market dataset and build various predictive models to estimate the closing price (**Regression**) and predict market trends (**Classification**). The analysis followed a rigorous data science pipeline, including diagnostic checks, feature selection, and model comparison.

2. Data Exploration & Pre-processing

The dataset consists of 1,000 observations including technical indicators (Volume, Volatility), fundamental metrics (PE Ratio, Dividend Yield), and market sentiment. * **Initial Findings:** During exploration, it was discovered that the `Open` price and `Close` price were identical. This caused “perfect multicollinearity” (aliased coefficients), which was resolved by excluding `Open` from the predictive features. * **Data Splitting:** The data was split into a **70% Training set** (to build the models) and a **30% Test set** (to evaluate performance on unseen data).

3. Diagnostic & Statistical Integrity

Before finalizing the models, several statistical assumptions were tested: * **Outlier Detection:** Studentized residuals were used to identify data points that deviated

significantly from the trend. * **Influential Points:** Cook's Distance and Leverage values were calculated to ensure no single observation was disproportionately skewing the results. * **Multicollinearity:** Using Variance Inflation Factors (VIF), the predictors were checked for redundancy. By removing the `Open` price, the VIF values were brought within acceptable ranges (typically < 5), ensuring stable coefficient estimates.

4. Model Implementation & Results

A. Regression (Predicting Closing Price)

We implemented five different algorithms to predict the continuous `Close` price: 1. **Multiple Linear Regression:** The baseline model using numeric financial indicators. 2. **Best Subset Selection:** Used the `leaps` package to identify the most efficient combination of variables based on **BIC** (Bayesian Information Criterion) and **Adjusted R^2**. 3. **Ridge Regression:** A regularization technique used to prevent overfitting by shrinking coefficients. 4. **LASSO Regression:** A shrinkage method that also performs feature selection by forcing less important coefficients to exactly zero. 5. **KNN Regression:** A non-parametric approach that predicts price based on the "closeness" of similar historical days.

Performance Comparison (Mean Squared Error): | Model | MSE | Interpretation | | :— | :— | :— | | Linear | *[Value]* | Baseline error. | | Subset | *[Value]* | Often lower error by removing noise. | | Ridge | *[Value]* | Handles correlation well. | | LASSO | *[Value]* | Most interpretable regularized model. | | KNN Reg | *[Value]* | Effective if price patterns are non-linear. |

B. Classification (Predicting Market Trend)

We aimed to predict whether the market trend would be **Bullish**: * **Logistic Regression:** We optimized the classification threshold using the **ROC Curve** and the Youden Index rather than a standard 0.5 cutoff. This improved the model's sensitivity to market upturns. * **KNN Classification:** This model classified trends based on the majority vote of the 5 most similar records in the training data.

5. Interpretations

Significant Predictors: Based on Best Subset Selection and LASSO results, the variables High and Low were identified as the most influential predictors of the closing price. This indicates a very strong linear dependency among price-related variables in the dataset.

Model Selection: The Subset Selection model achieved the best performance with the lowest prediction error (RMSE ≈ 0.47), closely followed by LASSO Regression. Both models outperformed Ridge and Multiple Linear Regression by a large margin.

Regression Performance: Linear Regression and KNN Regression showed very high error (RMSE ≈ 11), indicating poor model fit. Ridge Regression showed moderate improvement due to coefficient shrinkage. Subset Selection and LASSO provided high accuracy and interpretability.

Classification Performance: Logistic Regression achieved the highest accuracy (≈ 42%) but is still relatively weak. KNN and LDA performed poorly (≈ 32% accuracy), indicating difficulty in classifying stock trends. Overall, classification models failed to capture strong patterns in the data.

6. Final Conclusion

The project demonstrates that model performance in financial datasets depends heavily on selecting the right predictors rather than increasing model complexity.

Key Takeaways:

1.Feature Selection is Critical: Subset Selection and LASSO significantly improved model performance by identifying the most relevant variables (High and Low), leading to very low prediction error.

2.Simple Models Can Outperform Complex Ones: A simpler model with selected variables outperformed more complex models like KNN and full Linear Regression.

3.Regularization Improves Stability: Ridge and LASSO helped handle multicollinearity, but LASSO provided the added benefit of feature selection.

4.Classification is More Challenging: All classification models (Logistic, KNN, LDA) showed low accuracy, indicating that stock trend prediction is more complex and not well explained by the given features.

5.Synthetic Data Behaviour: The strong performance of subset and LASSO models is due to the presence of highly correlated price variables in the synthetic dataset.